Hochschule Karlsruhe
University of
Applied Sciences

Rusts Konzept der Speichersicherheit

Seminararbeit
von

David Grandel

Fakultät für Informatik und Wirtschaftsinformatik

Betreuer: Prof. Dr. Carsten Sinz
Bearbeitungszeitraum: Sommersemester 2026

Zusammenfassung

Speicherzugriffsfehler wie Use After Free und Out of Bounds Write zählen zu den häufigsten Sicherheitsproblemen in Software und entstehen besonders leicht in C, weil Speicher manuell verwaltet wird. Diese Seminararbeit untersucht, wie Rust derartige Fehler durch Compilezeitprüfungen verhindert.

Zunächst werden beide Fehlertypen anhand minimaler C Beispiele erläutert. Anschließend wird Rusts Konzept aus Ownership, Borrowing, Lifetimes und dem Borrow Checker dargestellt und an denselben Beispielen nachvollzogen. Use After Free scheitert beim Kompilieren, Out of Bounds Zugriffe werden zur Compilezeit abgelehnt oder zur Laufzeit kontrolliert abgebrochen.

Rust macht zentrale Speicherfehler früh sichtbar oder verhindert sie vollständig, ohne Garbage Collection. Grenzen bleiben durch unsicheren Code und Migrationsaufwand in bestehenden C Codebasen. Praxisdaten aus der Android Entwicklung zeigen dennoch einen deutlichen Rückgang speicherbezogener Schwachstellen nach verstärkter Nutzung speichersicherer Sprachen.

Inhaltsverzeichnis

Zusammenfassung **i**

1 Einleitung **1**

2 Speichersicherheit als zentrales Sicherheitsproblem **2**
2.1 Erfassung und Bewertung von Schwachstellen 2
2.2 Begriff der Speichersicherheit 4
2.3 Use After Free 5
2.4 Out of Bounds Write 7
2.5 Sicherheitsrelevante Konsequenzen 9
2.6 Lösungsansätze 10

3 Speichersicherheit in Rust **12**
3.1 Ownership 12
3.2 Übertragung von Ownership 14
3.3 Borrowing 15
3.4 Lifetimes und Borrow Checker 17
3.5 Verhinderung von Use After Free 18
3.6 Grenzprüfung zur Compilezeit 20
3.7 Grenzprüfung zur Laufzeit 22
3.8 Formale Sicherheitsnachweise 23

4 Fazit **25**

Literaturverzeichnis **27**

1 Einleitung

Sicherheit ist ein zentrales Thema in der Softwareentwicklung. Angreifer entwickeln immer wieder neue Methoden, um Softwaresystemen Schaden zuzufügen, unerlaubten Zugriff auf Daten von Nutzern oder Firmen zu erhalten oder Dienste lahmzulegen. Dabei verändern sich Technologien in der Softwareentwicklung ständig, wodurch fortlaufend neue Möglichkeiten für Angriffe entstehen.

Um herauszufinden, welche Probleme am häufigsten Sicherheitslücken verursachen, landet man relativ schnell bei der Common Weakness Enumeration, kurz CWE [1]. In dieser Liste werden verschiedene Schwachstellen aufgelistet und definiert, die bestimmte Angriffsmöglichkeiten von Software beschreiben.

Die größte Schwachstelle laut der aktuellsten Statistik der CWE aus dem Jahr 2025 ist tatsächlich eine Injection Schwachstelle [2], genauer gesagt die OS Command Injection. Sie entsteht, wenn Entwickler eine Anwendung nicht ausreichend gegen bösartige Argumente absichern. Um diese Schwachstelle soll es in dieser Arbeit jedoch nicht gehen.

Platz zwei und drei im Ranking beschreiben Fehler, die mit der Speichersicherheit zusammenhängen. Ein Use After Free Bug landet auf Platz zwei des Rankings. Dabei handelt es sich um einen Programmierfehler, der dazu führt, dass über einen Dangling Pointer erneut auf Speicher zugegriffen wird, der bereits zur weiteren Nutzung freigegeben wurde. Ein Out of Bounds Write steht tatsächlich auf Platz drei, während dieser Fehler in der Statistik aus dem Jahr 2024 sogar Platz eins belegte [3]. Beim Out of Bounds Write wird Speicher geschrieben, der außerhalb des gültigen Bereichs eines Puffers liegt, typischerweise auf dem Stack. Solche Fehler treten besonders häufig in Programmiersprachen mit manueller Speicherverwaltung auf, allen voran in C. Dort liegt die Verwaltung des Speichers vollständig in der Hand des Entwicklers, wodurch eine fehlerhafte Nutzung schnell zu derartigen Schwachstellen führen kann.

Diese beiden Fehlerphänomene sollen im weiteren Verlauf im Zusammenhang mit Speichersicherheit genauer betrachtet werden. Im Fokus steht das Lösungskonzept, das Rust anbietet, um derartige schwerwiegende Speicherfehler von Grund auf zu vermeiden. Die Theorie, die hinter dem Konzept der Speichersicherheit in Rust steckt, sowie kleine Beispiele dazu sollen in dieser Arbeit gezeigt werden.

2 Speichersicherheit als zentrales Sicherheitsproblem

Bevor die Speichersicherheitsprobleme im Detail betrachtet werden, lohnt sich ein Blick auf die zugrunde liegende Statistik. Dafür wird zunächst erläutert, wie die CWE Schwachstellen erfasst, bewertet und in Kategorien einteilt. So lässt sich nachvollziehen, welche Bedeutung Speichersicherheit in den Auswertungen tatsächlich einnimmt. Anschließend wird der Begriff Speichersicherheit präzisiert. Die beiden zuvor genannten Schwachstellen werden anhand kurzer C Beispiele nachvollzogen. Dabei wird Schritt für Schritt gezeigt, wie Use After Free und Out of Bounds Write in der Praxis entstehen können und warum solche Fehler in Sprachen mit manueller Speicherverwaltung besonders leicht auftreten. Zum Abschluss des Kapitels werden die sicherheitsrelevanten Konsequenzen dieser Fehler betrachtet. Darauf aufbauend werden gängige Lösungsansätze vorgestellt, um damit den Übergang zum folgenden Kapitel vorzubereiten, in dem Rusts Ansatz zur Compilezeitprüfung von Speichersicherheit im Mittelpunkt steht.

2.1 Erfassung und Bewertung von Schwachstellen

In der Einleitung wurde bereits die Common Weakness Enumeration (CWE) eingeführt, eine Sammlung, die verschiedene Softwareschwachstellen auflistet und definiert. Um vergleichbare Fehlerklassen zu erfassen, werden abstrakte Schwachstellen definiert und auf tatsächlich dokumentierte Sicherheitslücken abgebildet. Diese Zuordnung geht auf die Idee einer gemeinsamen Schwachstellenkatalogisierung zurück [4].

Die dokumentierten Einzelfälle werden als Common Vulnerabilities and Exposures (CVEs) bezeichnet. Sie stammen aus einer öffentlich zugänglichen Datenbank, in der Softwarefehler erfasst werden. Ein CVE kann eingereicht werden, wenn eine Schwachstelle beschrieben und einer CVE Nummer zugeordnet werden kann. Die Vergabe erfolgt über sogenannte CVE Numbering Authorities (CNAs), zu denen unter anderem große Technologieunternehmen wie Microsoft und Google gehören [5]. Die Datenbasis ist nicht vollständig, da nicht jeder Fehler veröffentlicht wird. Aufgrund der breiten Beteiligung gilt sie dennoch als eine der repräsentativsten frei zugänglichen Quellen für Softwareschwachstellen.

Use After Free und Out of Bounds Write aus der Einleitung teilen gemeinsame Merkmale mit zahlreichen weiteren Schwachstellen in der CWE Bewertung. Die betrachtete Auswertung bezieht sich auf Known Exploited Vulnerabilities (KEV), also nachweislich ausgenutzte Schwachstellen. Dafür werden Schwachstellen in Kategorien eingeteilt. Abbildung 1 zeigt die Verteilung der zehn am häufigsten ausgenutzten KEV Schwach-

stellen nach Kategorie. Speichersicherheit (Memory Safety) nimmt mit 31 Prozent den zweitgrößten Anteil ein, direkt hinter Injection mit 33 Prozent. Weitere Kategorien wie Access Control, Resource Control und File Handling liegen deutlich zurück.

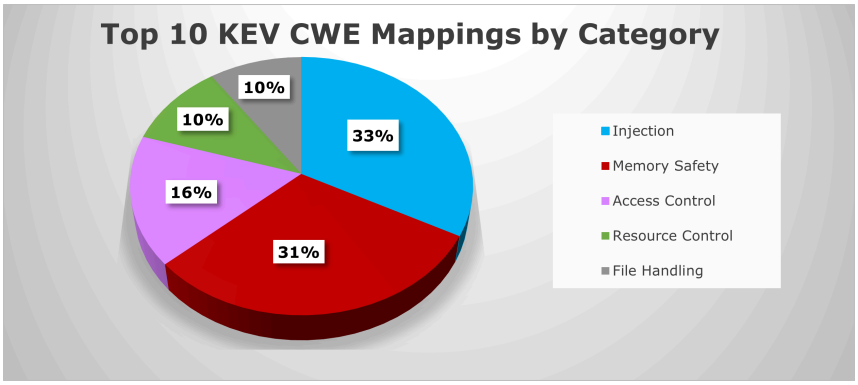


Abbildung 1 : Verteilung der CWE Top 10 KEV Schwachstellen nach Kategorie [6]

Zur Gewichtung einzelner Schwachstellen dient der sogenannte Danger Score [5]. Er kombiniert die Häufigkeit einer Schwachstelle im Datensatz mit der durchschnittlichen Schwere bei Ausnutzung. Die Häufigkeit entspricht der Anzahl der CVE Einträge, denen die Schwachstelle als Ursache zugeordnet ist. Die Schwere wird über den Common Vulnerability Scoring System (CVSS) Basiswert abgebildet. Beide Größen werden auf einen Wertebereich zwischen 0 und 1 normiert. Der Danger Score ergibt sich daraus wie folgt:

$$\text{Danger Score} = F_r \cdot S_v \cdot 100$$

$$F_r = \frac{\text{Anzahl(CWE)} - \text{Anzahl}_{\text{min}}}{\text{Anzahl}_{\text{max}} - \text{Anzahl}_{\text{min}}}$$

$$S_v = \frac{\overline{\text{CVSS}}(\text{CWE}) - \text{CVSS}_{\text{min}}}{\text{CVSS}_{\text{max}} - \text{CVSS}_{\text{min}}}$$

Dabei bezeichnet F_r die normalisierte Häufigkeit einer Schwachstelle und S_v die normalisierte durchschnittliche CVSS Schwere [5]. Anzahl(CWE) ist die Anzahl der CVE Zuordnungen, $\overline{\text{CVSS}}(\text{CWE})$ der durchschnittliche CVSS Basiswert. Die Indizes min und max bezeichnen jeweils den kleinsten und größten Wert im Datensatz. F_r und S_v liegen damit jeweils zwischen 0 und 1. Schwachstellen, die selten auftreten oder bei Ausnutzung geringe Auswirkungen haben, erhalten einen niedrigen Score. Häufige Schwachstellen mit schwerwiegenden Folgen werden entsprechend höher bewertet.

2.2 Begriff der Speichersicherheit

Ausgehend von der CWE Bewertung stellt Speichersicherheit ein zentrales Sicherheitsproblem dar. Damit ist jedoch noch nicht geklärt, was unter Speichersicherheit, im Englischen Memory Safety, konkret zu verstehen ist. Der Begriff wird häufig verwendet, eine einheitliche Definition ist aber weniger selbstverständlich, als es zunächst wirkt.

Grundsätzlich lässt sich zwischen der Speichersicherheit einer Programmausführung, eines Programms und einer Programmiersprache unterscheiden [7]. Eine Programmausführung gilt als speichersicher, wenn während ihres Ablaufs kein Speicherzugriffsfehler auftritt. Ein Programm gilt als speichersicher, wenn alle möglichen Ausführungen speichersicher sind. Entsprechend gilt eine Programmiersprache als speichersicher, wenn jedes in ihr erzeugbare Programm speichersicher ist.

Eine verbreitete Definition orientiert sich an einer Liste typischer Speicherzugriffsfehler [7], [8]. Solche Fehler zählen zu den ältesten und persistierendsten Sicherheitsproblemen in Software, die in Sprachen wie C oder C++ geschrieben ist. Fehlt Speichersicherheit, können Angreifer das Verhalten von Programmen verändern oder die Kontrolle über den Programmablauf übernehmen. Trotz zahlreicher Schutzmechanismen bleiben Speicherfehler bis heute eine ernsthafte Bedrohung. Tabelle 1 fasst fünf zentrale Speicherzugriffsfehler zusammen. Tritt mindestens einer dieser Fehler in einer möglichen Ausführung auf, ist das Programm nicht speichersicher.

Speicherzugriffsfehler	Zugriffsart	Beschreibung
Out of Bounds Write	Schreiben	Schreibzugriff außerhalb eines gültigen Puffers oder Arrays
Out of Bounds Read	Lesen	Lesezugriff außerhalb eines gültigen Puffers oder Arrays
Use After Free	Lesen oder Schreiben	Zugriff auf Speicher, der bereits freigegeben wurde
Use of Uninitialized Memory	Lesen oder Schreiben	Zugriff auf reservierten, aber noch nicht initialisierten Speicher
Double Free	Freigabe	Erneute Freigabe eines bereits freigegebenen Speicherbereichs

Tabelle 1 : Fünf zentrale Speicherzugriffsfehler [8]

Kurz gefasst lässt sich Speichersicherheit auch so erklären: Ein Programm ist speichersicher, solange es keinen undefinierten Speicher verwendet [7]. Undefinierter Speicher ist Speicher, der nie alloziert wurde oder dessen Lebensdauer bereits beendet ist. Explizit alloziert wird Speicher auf dem Heap, auf dem Stack oder im statischen Datenbereich. Die genannten Speicherzugriffsfehler lassen sich als unzulässige Zugriffe auf undefinierten Speicher interpretieren.

2.3 Use After Free

Use After Free bezeichnet einen Speicherzugriffsfehler, bei dem auf Speicher zugegriffen wird, der bereits freigegeben wurde. Der Fehler tritt ausschließlich im Zusammenhang mit dynamischer Speicherverwaltung auf. Betroffen ist damit Speicher, der zur Laufzeit auf dem Heap alloziert wurde.

Im Gegensatz zum Stack, auf dem lokale Variablen und Funktionsparameter automatisch verwaltet werden, dient der Heap für Speicher, dessen Lebensdauer über den aktuellen Funktionsaufruf hinausgehen soll. Allokation und Freigabe erfolgen in C explizit über Funktionen wie `malloc()` und `free()`. Der Heap ist ein großer Speicherbereich, aus dem der Laufzeitallocator bei Bedarf einzelne Blöcke bereitstellt. Freigegebene Blöcke können bei späteren Allokationen erneut vergeben werden. Diese Flexibilität ermöglicht effiziente Speichernutzung, birgt aber das Risiko, dass Zeiger auf bereits freigegebenen Speicher weiterhin existieren. Das folgende C Beispiel veranschaulicht den Fehler, da C zusammen mit C++ zu den verbreitetsten Sprachen mit manueller Speicherverwaltung zählt.

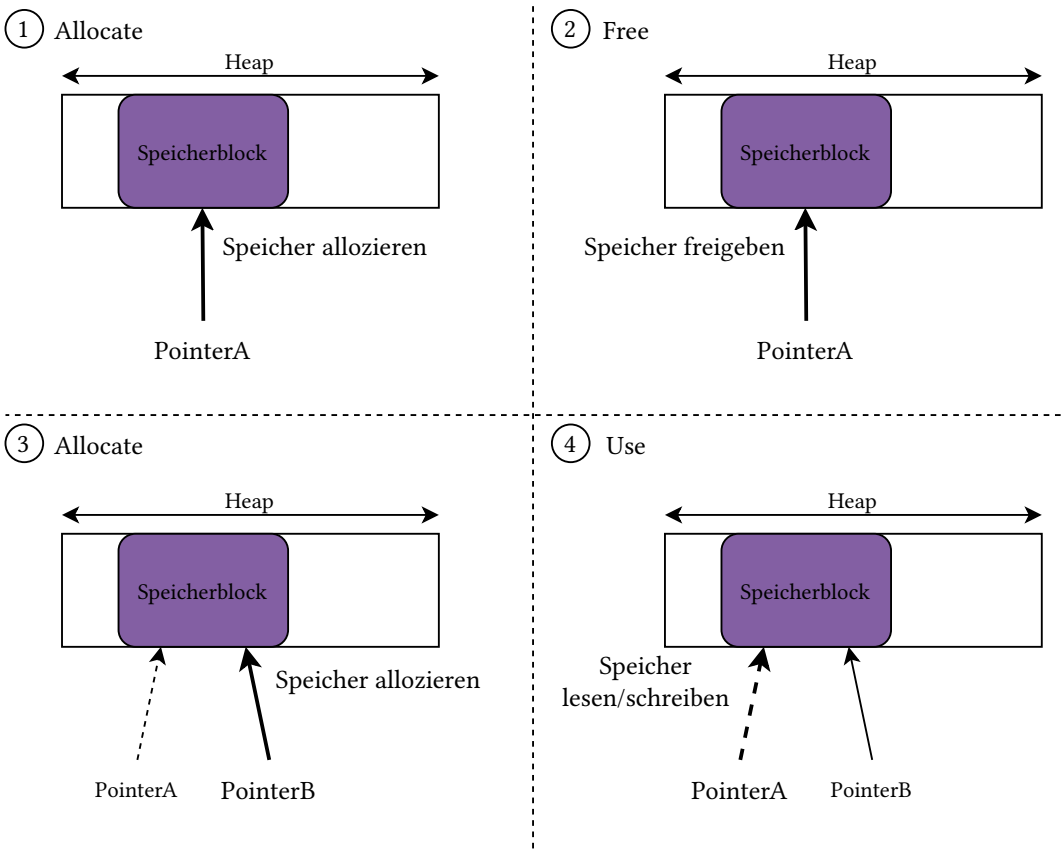


Abbildung 2 : Vierphasiger Ablauf eines Use After Free Fehlers auf dem Heap [9]

Nach der CWE Definition greift ein Use After Free vor, wenn ein Programm Speicher erneut verwendet oder referenziert, nachdem dieser freigegeben wurde [9]. Zu einem

späteren Zeitpunkt kann derselbe Speicherbereich erneut alloziert. Der ursprüngliche Zeiger verweist dabei weiterhin auf eine Adresse innerhalb der neuen Allokation. Zugriffe über den ursprünglichen Zeiger sind ungültig, da der Speicherbereich faktisch dem Code gehört, der über den neuen Zeiger arbeitet. Ein Zeiger, der nach einer Freigabe nicht mehr auf gültigen Speicher verweist, wird als Dangling Pointer bezeichnet. Typischerweise durchläuft ein Use After Free vier Phasen, die in Abbildung 2 zusammengefasst sind:

1. **Allocate:** Speicher wird auf dem Heap alloziert.
2. **Free:** Der Speicher wird freigegeben. Bestehende Zeiger verweisen danach auf ungültigen Speicher und werden zu Dangling Pointern.
3. **Allocate:** Derselbe Speicherbereich kann erneut alloziert werden.
4. **Use:** Ein Zugriff über den Dangling Pointer liest oder schreibt fälschlich den neu zugewiesenen Inhalt.

Quelltext 1 zeigt ein minimales Beispiel. In Zeile 5 wird über `malloc()` Speicher für einen Integer alloziert und `buffer_ptr1` gesetzt. Zeile 6 schreibt den Wert 21 in diesen Block. Zeile 8 erzeugt mit `buffer_ptr2` einen zweiten Zeiger auf dieselbe Adresse, ohne neuen Speicher zu reservieren. Mit `free(buffer_ptr1)` in Zeile 10 wird der Block freigegeben. Damit endet die Gültigkeit der Allokation. Beide Zeiger enthalten weiterhin die bisherige Adresse und sind ab diesem Punkt Dangling Pointer. Entspricht dies der Free Phase aus Abbildung 2.

```
1  #include <stdlib.h>
2  #include <stdio.h>
3
4  int main(int argc, char **argv) {
5      int *buffer_ptr1 = malloc(sizeof(int));
6      *buffer_ptr1 = 21;
7
8      int *buffer_ptr2 =  buffer_ptr1;
9
10     free(buffer_ptr1);
11
12     int *buffer_ptr3 = malloc(sizeof(int));
13     *buffer_ptr3 = 42;
14
15     printf("%d", *buffer_ptr2);
16 }
17
```

Quelltext 1 : Minimales Use After Free Beispiel in C

Zeile 12 alloziert erneut Speicher für einen Integer, was der zweiten Allocate Phase entspricht. Je nach Allocator und Heap Zustand kann `buffer_ptr3` dieselbe Adresse erhalten wie zuvor `buffer_ptr1`. Zeile 13 überschreibt den Inhalt mit 42.

Zeile 15 dereferenziert `buffer_ptr2` und gibt den Wert aus. Der Zugriff erfolgt über einen Dangling Pointer und entspricht der Use Phase. Statt des ursprünglichen Werts 21 wird der Wert 42 ausgegeben, der über `buffer_ptr3` geschrieben wurde. Der Fehler entsteht, weil zwei Zeiger auf denselben Speicher verwiesen, die Freigabe aber nur über einen von ihnen durchgeführt wurde.

Dieses Verhalten ist sicherheitskritisch, wenn über den neuen Zeiger vertrauliche Daten abgelegt werden und der Dangling Pointer unberechtigten Lesezugriff ermöglicht. Derartige Fehler entstehen besonders leicht, wenn mehrere Zeiger auf denselben Heap Block verweisen und die Freigabe nicht für alle Zeiger nachvollziehbar ist.

2.4 Out of Bounds Write

Neben Use After Free zählt Out of Bounds Write zu den häufigsten Speicherzugriffsfehlern aus der Einleitung. Anders als Use After Free betrifft dieser Fehler nicht nur den Heap, sondern tritt besonders häufig auf dem Stack auf. Der Stack dient der Verwaltung von Funktionsaufrufen in Programmen. Bei jedem Aufruf werden lokale Variablen, Funktionsparameter und Rücksprunginformationen in einem last in, first out Verfahren abgelegt. Beim Verlassen einer Funktion wird der zugehörige Stackbereich wieder freigegeben. Im Gegensatz zum Heap erfolgt diese Verwaltung automatisch durch den Compiler und die Laufzeitumgebung. Typischerweise wächst der Stack dabei in Richtung abnehmender Speicheradressen. Welche Variablen an welcher Adresse liegen, hängt vom Compiler und der Zielplattform ab. Gerade diese feste Nachbarschaft von Daten auf dem Stack macht Out of Bounds Write besonders gefährlich.

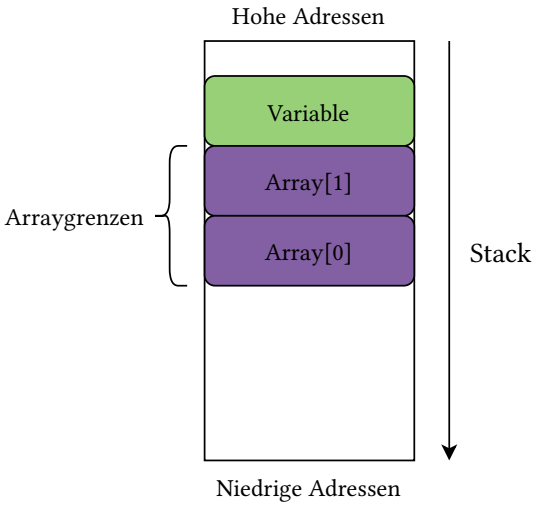


Abbildung 3 : Out of Bounds Write auf dem Stack [10]

Nach der CWE Definition liegt ein Out of Bounds Write vor, wenn Daten hinter das Ende oder vor den Anfang eines vorgesehenen Puffers geschrieben werden [10]. Schreibt ein Programm außerhalb der gültigen Grenzen eines Arrays oder Puffers, kann benachbarter Speicher überschrieben werden. In der CWE Terminologie wird der daraus resultierende Schaden häufig als Memory Corruption bezeichnet, also die Beschädigung von Speicher durch ungültige Schreibzugriffe [10]. Abbildung 3 veranschaulicht den Fehler am Beispiel eines Stacklayouts. Dort liegen eine lokale Variable und ein Array unmittelbar hintereinander. Gültige Indizes des Arrays reichen von 0 bis 1. Wird dennoch über Index 2 geschrieben, trifft der Schreibzugriff nicht mehr den Arraybereich, sondern die benachbarte Variable. Der ursprüngliche Wert der Variable wird so stillschweigend überschrieben.

```c
#include <stdio.h>

void write_arr_at_index(int *arr, int index) {
    arr[index] = 99;
}

int main() {
    int var = 5;
    int array[2] = {2, 2};

    write_arr_at_index(array, 2);

    printf("%d", var);
}

```

Quelltext 2 : Minimales Out of Bounds Write Beispiel in C

Quelltext 2 zeigt ein minimales C Beispiel. In Zeile 8 wird die Variable `var` mit dem Wert 5 initialisiert. Zeile 9 legt ein Array mit zwei Elementen an. Zeile 11 ruft `write_arr_at_index(array, 2)` auf und übergibt den Index 2. Für ein Array der Länge 2 sind jedoch nur die Indizes 0 und 1 gültig. Die Funktion in Zeile 3 und 4 führt den Schreibzugriff ohne Prüfung des Index aus. Entsprechend der in Abbildung 3 dargestellten Anordnung kann der Zugriff auf `arr[2]` den Speicher der Variable `var` treffen. Beim GNU C Compiler (GCC) liegen lokale Variablen und Arrays auf dem Stack typischerweise in unmittelbarer Nachbarschaft mit absteigenden Adressen. Zeile 13 gibt daher nicht den erwarteten Wert 5 aus, sondern 99.

Auch wenn die Veränderung einer einzelnen Variable harmlos erscheint, sind die Folgen bei sicherheitskritischem Code gravierend. Liegt an der betroffenen Stackposition statt einer einfachen Variable beispielsweise eine Rücksprungadresse oder ein Funktion-

szeiger, kann ein gezielter Out of Bounds Write die Programmausführung umleiten und die Ausführung von Angreifercode ermöglichen [8].

2.5 Sicherheitsrelevante Konsequenzen

Die beiden beschriebenen Fehler wirken sich auf mehrere zentrale Sicherheitsziele aus. Neben Integrität und Verfügbarkeit sind vor allem Vertraulichkeit und die Kontrolle über die Programmausführung betroffen [8], [11].

Vertraulichkeit stellt sicher, dass Informationen nur für berechtigte Empfänger zugänglich sind. Use After Free kann dieses Ziel verletzen, wenn über Dangling Pointer in bereits freigegebenen Speicher gelesen wird. Der Speicherbereich kann inzwischen anderen Teilen des Programms zugeordnet sein, etwa einem neuen Heap Block mit vertraulichen Inhalten. In Quelltext 1 liest `buffer_ptr2` einen Wert aus, der eigentlich über `buffer_ptr3` geschrieben wurde. Übertragen auf realen Code entspricht das dem unbefugten Auslesen fremder Daten aus dem Prozessspeicher.

Integrität bedeutet, dass Daten nicht unbefugt verändert werden. Use After Free gefährdet dieses Ziel auf dem Heap, wenn über Dangling Pointer in bereits freigegebenen Speicher geschrieben wird. Out of Bounds Write bedroht die Integrität vor allem auf dem Stack. Wie in Quelltext 2 gezeigt, überschreibt ein ungültiger Index den Wert einer benachbarten Variable. Allgemeiner lassen sich damit benachbarte Speicherbereiche manipulieren, etwa Metadaten, Zeiger oder Kontrollstrukturen.

Neben der Integrität einzelner Datenwerte kann Speicherkorruption die Kontrolle über den Programmablauf selbst gefährden. Out of Bounds Write erlaubt es, Funktionszeiger oder Rücksprungadressen auf dem Stack zu überschreiben. Use After Free kann zusammen mit weiteren Techniken dazu genutzt werden, unerwartete Ausführungspfade zu erzeugen [8]. Die CWE ordnet Out of Bounds Write ausdrücklich der Folge Execute Unauthorized Code or Commands zu [10]. Damit lassen sich Speicherfehler von reinen Datenfehlern zu Angriffen eskalieren, die die ursprüngliche Logik des Programms vollständig unterlaufen.

Verfügbarkeit beschreibt, ob ein System oder Dienst für berechtigte Nutzer erreichbar bleibt. Speicherfehler können diese Verfügbarkeit durch Abstürze oder instabiles Verhalten einschränken [11]. Bei Use After Free spielt die interne Verwaltung des Heaps eine Rolle. Speicherallocator wie die Implementierung von `malloc` in der GNU C Library führen nach einer Freigabe häufig eine Chunk Konsolidierung durch, bei der benachbarte freie Blöcke zusammengeführt werden. Greift ein Programm danach über einen Dangling Pointer erneut auf den freigegebenen Bereich zu, können die internen Verwaltungsinformationen des Heaps nicht mehr gültig sein. Der Prozess bricht in solchen Fällen oft ab. Out of Bounds Write kann die Verfügbarkeit ebenfalls beeinträchtigen. Wird etwa eine Rücksprungadresse auf dem Stack überschrieben, kehrt die Ausführung nicht mehr kontrolliert in den aufrufenden Code zurück. Auch ungültige Zugriffe auf nicht vorgesehene Speicherbereiche können den Prozess beenden. Für netzwerk- oder

serverbasierte Anwendungen bedeutet ein solcher Absturz unmittelbar den Ausfall des betroffenen Dienstes.

2.6 Lösungsansätze

Die gezeigten Beispiele verdeutlichen, warum Speicherfehler in C und C++ ein dauerhaftes Sicherheitsproblem darstellen. Gegen Speicherfehler haben sich in der Praxis im Wesentlichen drei Ansätze etabliert. Sie unterscheiden sich darin, ob Speicher zur Laufzeit automatisch verwaltet, zur Laufzeit überwacht oder bereits zur Compilezeit geprüft.

Der verbreitetste Ansatz ist der Einsatz speichersicherer Programmiersprachen mit automatischer Speicherverwaltung. Dabei übernimmt eine Laufzeitumgebung die Verwaltung des Heaps. Ein Garbage Collector ermittelt zur Laufzeit, welche Speicherobjekte noch erreichbar sind, und gibt nicht mehr benötigten Speicher automatisch frei. Entwickler müssen in solchen Sprachen typischerweise weder `malloc()` noch `free()` aufrufen und können damit Use After Free oder Double Free in der vorliegenden Form vermeiden. Sprachen wie Java, Python oder JavaScript setzen diesen Ansatz unterschiedlich um, gelten aber gemeinsam als speichersicherer als C oder C++. Häufig werden zusätzlich Laufzeitprüfungen für Arraygrenzen eingesetzt, um Out of Bounds Zugriffe abzufangen. Der Preis dafür ist ein zusätzlicher Laufzeitaufwand, der vor allem durch den Garbage Collector entsteht. In performancekritischen Systemen kann dieser Overhead spürbar sein. Für bestehende C Codebasen ist ein vollständiger Wechsel auf eine andere Sprache zudem oft mit erheblichem Migrationsaufwand verbunden [12].

Ein zweiter Ansatz zielt auf bestehenden C oder C++ Code. Dabei wird das Programm beim Kompilieren oder zur Laufzeit so erweitert, dass ungültige Speicherzugriffe erkannt werden, bevor sie unkontrolliert eskalieren. AddressSanitizer (ASan) ist ein verbreitetes Beispiel für diese Kategorie. ASan fügt zusätzliche Prüfungen ein und nutzt sogenanntes Shadow Memory, um nachzuvollziehen, welche Speicheradressen gültig sind. Das Werkzeug findet unter anderem Out of Bounds Zugriffe und Use After Free während der Programmausführung. Laut den Autoren entsteht dabei im Mittel eine Verlangsamung von etwa 73 Prozent [13]. Valgrind verfolgt einen ähnlichen Zweck, arbeitet aber durch dynamische Instrumentierung zur Laufzeit und ist deutlich langsamer. Es eignet sich vor allem für Tests und Fehlersuche in der Entwicklung, nicht für den dauerhaften Produktionseinsatz. Entscheidend ist, dass diese Werkzeuge Fehler erst zur Laufzeit sichtbar machen. Sie verhindern Speicherfehler nicht vollständig und decken nicht alle Fehlerklassen zuverlässig ab [8]. Auch Rust führt bei einigen Prüfungen, etwa Grenzchecks bei Arrayzugriffen, zur Laufzeit einen kontrollierten Abbruch aus, statt undefiniertes Verhalten zu erzeugen. Die zentrale Speichersicherheit wird in Rust jedoch nicht über Instrumentierung, sondern über Compilezeitprüfungen erreicht.

Der dritte Ansatz verschiebt die Erkennung von Speicherfehlern in den Compiler. Potenziell unsichere Programme werden gar nicht erst gebaut, wenn sie gegen die Regeln der Sprache verstoßen. Damit lassen sich Use After Free, Dangling Pointer und viele

Out of Bounds Zugriffe bereits vor der Ausführung ausschließen. Im Idealfall entsteht dafür kein dauerhafter Garbage Collection Overhead, weil die Speicherverwaltung statisch geprüft wird statt vollständig zur Laufzeit nachverfolgt zu werden. Dieser Ansatz erfordert jedoch strengere Sprachregeln und ein Umdenken bei der Programmierung. Rust verfolgt genau diese Strategie mit Konzepten wie Ownership und Borrowing. Im folgenden Kapitel wird dargestellt, wie Rust Speicherfehler zur Compilezeit verhindert und damit einen alternativen Weg zu Garbage Collection und nachträglicher Instrumentierung eröffnet.

3 Speichersicherheit in Rust

Der Lösungsansatz, durch eine Prüfung zur Compile-Zeit Speicherfehler zu verhindern, ist genau die Strategie, die Rust verfolgt. Die Sprache zielt darauf ab, zwei Anforderungen zu vereinen, die in der Praxis lange als schwer vereinbar galten: direkten Zugriff auf Hardware und Speicher einerseits sowie Sicherheit und ein produktives Entwicklungserlebnis andererseits. Statt Speicherfehler erst zur Laufzeit zu entdecken oder sie durch einen Garbage Collector zu kompensieren, verschiebt Rust zentrale Sicherheitsprüfungen in den Compiler.

Rust ist dabei nicht nur eine Sprachdefinition, sondern ein umfassendes Ökosystem. Neben dem Compiler `rustc` gehören dazu unter anderem `cargo` als Build System und Paketmanager, `rustfmt` zur einheitlichen Formatierung von Quellcode sowie `rust-analyzer` als Language Server für Editoren. Diese Werkzeuge sind standardmäßig Teil der Rust Installation und sollen typische Entwicklungsaufgaben von der Projekterstellung bis zur statischen Analyse abdecken. Ob das Versprechen der Sprache in der Praxis trägt, lässt sich auch an der Beliebtheit bei den Entwicklern ablesen.

Laut dem Stack Overflow Developer Survey 2025 zählt Rust erneut zu den am meisten geschätzten Programmiersprachen [14]. Unter den Befragten, die in Rust bereits umfangreich entwickelt haben, gaben 72 Prozent an, auch im folgenden Jahr damit arbeiten zu wollen. Damit liegt Rust in diesem Ranking an erster Stelle. Zugleich erscheint Rust regelmäßig unter den Technologien, deren weiterer Einsatz besonders gewünscht wird. Solche Umfragen ersetzen keine technische Bewertung, verdeutlichen aber, dass Rust nicht nur als theoretisches Konzept wahrgenommen wird.

Entscheidend für diese Arbeit ist jedoch nicht die Popularität, sondern der Mechanismus hinter der Speichersicherheit. Nach den im vorherigen Kapitel betrachteten Fehlern Use After Free und Out of Bounds Write stellt sich die Frage, welche Sprachkonzepte Rust bereitstellt, damit derartige Fehler sicher erkannt oder gar nicht erst kompiliert werden. Im folgenden wird Schritt für Schritt dargestellt, wie Ownership, Borrowing und Lifetimes zusammenwirken, wie der Borrow Checker Dangling References zur Compilezeit ausschließt und wie Rust zusätzlich Grenzverletzungen bei Arrayzugriffen behandelt. Dabei werden die zuvor gezeigten C Beispiele in Rust nachgebaut, um die Unterschiede im konkreten Verhalten sichtbar zu machen.

3.1 Ownership

Das erste zentrale Konzept ist Ownership. Es beschreibt ein Regelwerk, nach dem Rust Speicher verwaltet. Ownership legt fest, welche Variable für welchen Wert im Speicher

verantwortlich ist. Der Compiler prüft diese Regeln zur Compilezeit. Verstöße führen zu Kompilierfehlern, nicht zu undefiniertem Verhalten zur Laufzeit. Damit unterscheidet sich Rust sowohl von Sprachen mit Garbage Collector als auch von C, in dem Allokation und Freigabe explizit erfolgen.

Ownership betrifft vor allem Daten auf dem Heap, deren Größe zur Compilezeit nicht feststeht. Entscheidend ist, dass der Compiler nachvollziehen kann, wann Speicher nicht mehr benötigt wird, und die Freigabe automatisch einplant. Tabelle 2 fasst die drei zentralen Ownership Regeln zusammen.

Regel	Inhalt
1	Jeder Wert in Rust hat genau einen Owner, also einen Besitzer.
2	Es kann zu jedem Zeitpunkt nur einen Owner geben.
3	Verlässt der Owner seinen Gültigkeitsbereich, wird der Wert verworfen.

Tabelle 2 : Die drei Ownership Regeln in Rust [15]

Regel 1 und 2 verhindern, dass mehrere Stellen im Programm gleichzeitig für dieselbe Heap Allokation verantwortlich sind. Damit entfällt die Situation aus dem C Beispiel, in der zwei Zeiger auf denselben Block zeigen, die Freigabe aber nur einmal erfolgt. Regel 3 sorgt dafür, dass Speicher automatisch freigegeben wird, sobald der zugehörige Owner nicht mehr erreichbar ist.

Ein Gültigkeitsbereich (Scope) ist der Programmabschnitt, in dem eine Variable gültig ist. Sie beginnt mit der Deklaration und endet, wenn der Scope verlassen wird. Regel 3 greift in mehreren typischen Situationen:

1. **Ende eines Blocks:** Schließt ein Block mit geschweiften Klammern, enden die darin deklarierten Variablen. Ihre Werte werden verworfen.
2. **Neue Zuweisung an denselben Owner:** Wird einer besitzenden Variable ein neuer Wert zugewiesen, wird der bisherige Wert sofort verworfen, noch bevor der neue Wert übernommen wird.
3. **Übergabe an eine Funktion:** Wird ein Wert an eine Funktion übergeben, kann die Ownership dabei in den Funktionsaufruf wechseln. Verlässt der Wert den ursprünglichen Scope, wird er dort verworfen. Die genaue Semantik dieser Übergabe wird im folgenden Abschnitt behandelt.

Verlässt ein Owner seinen Gültigkeitsbereich, ruft Rust automatisch die Funktion `drop()` auf. Für Typen wie `String` oder `Box<T>` enthält `drop()` die Logik zur Freigabe des Heap Speichers. Das Muster entspricht dem in C++ bekannten Konzept Resource Acquisition Is Initialization (RAII), bei dem Ressourcen am Ende ihrer Lebensdauer automatisch freigegeben werden [15].

Für die Speichersicherheit ergeben sich daraus zwei unmittelbare Vorteile gegenüber C. Erstens muss Heap Speicher nicht mehr manuell mit `free()` freigegeben werden. Der Compiler fügt die Freigabe an den korrekten Stellen ein. Zweitens kann eine Freigabe

nicht vergessen werden, solange das Programm kompiliert. Vergessene `free()` Aufrufe und damit verbundene Speicherlecks entfallen in dieser Form. Ob auch Use After Free vollständig ausgeschlossen ist, hängt zusätzlich davon ab, wie Ownership zwischen Variablen übertragen und temporär ausgeliehen wird. Darauf gehen die folgenden Abschnitte ein.

3.2 Übertragung von Ownership

Regel 2 verlangt, dass es zu jedem Zeitpunkt nur einen Owner gibt. Die Frage ist, wie sich Ownership von einer Variable auf eine andere überträgt, wenn Heap Daten beteiligt sind. Bei Werten fester Größe auf dem Stack ist der Fall unkritisch. Wird etwa einer zweiten Variablen ein Integer zugewiesen, wird der Wert kopiert. Danach existieren zwei unabhängige Variablen mit demselben Inhalt. Beide können ihren Gültigkeitsbereich unabhängig verlassen, ohne Konflikte bei der Speicherfreigabe.

Bei Heap Allokationen ist die Situation anders. Quelltext 3 zeigt ein minimales Beispiel mit `Box::new(42)`. `Box<T>` ist ein Smart Pointer, der einen Wert auf dem Heap hält. In Zeile 2 wird `pointer_1` zum Owner dieses Blocks. Zeile 3 überträgt die Ownership auf `pointer_2`. Entscheidend ist, was danach mit `pointer_1` geschieht. Im Gegensatz zu C entsteht hier kein zweiter Zeiger auf dieselbe Adresse. Stattdessen wird die Ownership verschoben.

```
1 fn main() {
2     let pointer_1 = Box::new(42);
3     let pointer_2 = pointer_1;
4 }
5
```

Quelltext 3 : Übertragung von Ownership mit `Box<T>` in Rust

Grundsätzlich lassen sich drei Varianten der Übergabe unterscheiden, wenn `pointer_1` seine Besitzrechte an `pointer_2` abgeben soll. Abbildung 4 stellt sie für einen `i32` Wert mit dem Inhalt 42 dar, der auf dem Heap liegt.

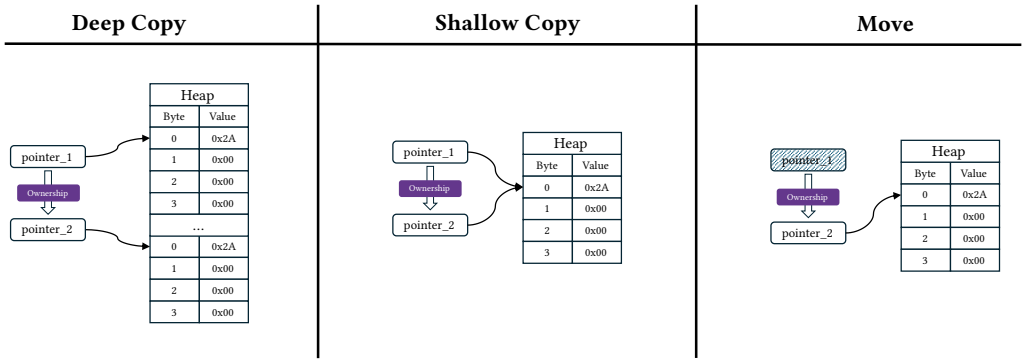


Abbildung 4 : Deep Copy, Shallow Copy und Move bei der Ownership Übergabe [15]

Bei einem **Deep Copy** wird der gesamte Inhalt der Heap Allokation dupliziert. Zusätzlich entsteht ein zweiter Zeiger auf den neuen Block. Beide Variablen besitzen danach jeweils eigenen Speicher mit identischem Inhalt.

Bei einem **Shallow Copy** wird nur der Zeiger kopiert. `pointer_1` und `pointer_2` verweisen dann auf dieselbe Heap Adresse. Der Inhalt wird nicht dupliziert. Diese Variante entspricht dem Verhalten aus dem C Beispiel in Quelltext 1, in dem `buffer_ptr2` auf denselben Block zeigt wie `buffer_ptr1`.

Bei einem **Move** wird der Zeiger zunächst ebenfalls übertragen. Im Unterschied zum Shallow Copy wird `pointer_1` danach ungültig. Es existiert weiterhin nur ein Owner. Der ausgegraute Eintrag in Abbildung 4 symbolisiert diese Ungültigkeit.

Beide Copy Varianten erhöhen die Zahl der Zeiger auf Heap Speicher. Beim Freigeben muss nachvollziehbar sein, welcher Zeiger verantwortlich ist und wann `drop()` aufgerufen wird. Andernfalls drohen Speicherlecks oder doppelte Freigabe. Ein Shallow Copy ist besonders fehleranfällig, weil mehrere Zeiger auf denselben Block zeigen können. Gibt einer der Owner den Speicher frei, werden die verbleibenden Zeiger zu Dangling Pointern. Genau dieses Muster begünstigt Use After Free in C.

Rust wählt für Heap Typen standardmäßig **Move Semantics** [15]. Nach `let pointer_2 = pointer_1` gilt `pointer_1` als bewegt und darf nicht mehr verwendet werden. Der Compiler lehnt weiteren Zugriff ab. Damit bleibt Regel 2 erhalten und Double Free wird vermieden, weil nur `pointer_2` beim Verlassen des Scopes `drop()` auslöst.

Kopieren ist in Rust weiterhin möglich, aber nur bewusst und explizit. Dafür nutzt Rust sogenannte Traits. Ein Trait beschreibt Verhalten, das ein Typ bereitstellen kann, vergleichbar mit einer Schnittstelle in anderen Sprachen. Ob ein Typ bei Zuweisung kopiert oder verschoben wird, hängt davon ab, welche Traits er implementiert.

Der Aufruf von `.clone()` erzeugt einen Deep Copy, sofern der Typ das `Clone` Trait implementiert. Für `Box<T>` ist das gegeben. Der Preis ist der zusätzliche Speicher und die Laufzeit für das Duplizieren. Typen mit dem `Copy` Trait werden hingegen stillschweigend kopiert, statt bewegt zu werden. Das gilt für einfache Werte fester Größe auf dem Stack [15]. Heap Typen wie `Box<T>` implementieren `Copy` nicht, weil sie Ressourcen freigeben müssen und das `Drop` Trait nutzen. Deshalb greift bei ihnen standardmäßig Move Semantics.

3.3 Borrowing

Move Semantics löst das Problem mehrerer Owner auf dem Heap, schränkt aber den Zugriff ein. Soll ein Wert in einer Funktion genutzt werden, ohne die Ownership abzugeben, reicht reines Verschieben nicht aus. Typischerweise müsste der Wert nach dem Aufruf zurückgegeben werden, damit er im aufrufenden Code weiter existiert. Genau das vermeidet Rust mit Referenzen und Borrowing.

Eine Referenz ist eine Adresse, über die auf einen Wert zugegriffen werden kann, ohne diesen zu besitzen [16]. Im Gegensatz zu einem rohen Zeiger in C garantiert Rust, dass eine Referenz während ihrer Gültigkeit auf einen des Wert passenden Typs zeigt. Das Anlegen einer Referenz heißt Borrowing. Der Owner bleibt unverändert und es gibt weiterhin genau einen Owner, zusätzlich existiert eine temporäre Zugriffsberechtigung.

Quelltext 4 zeigt das Prinzip anhand einer einfachen Funktion `increment`. In Zeile 1 bis 3 wird `value` als veränderbare Referenz auf einen `i32` deklariert. Das kaufmännische Und `&` kennzeichnet den Referenztyp. In Zeile 2 wird über den Dereferenzierungsoperator `*` der Wert am Ziel der Referenz erhöht. In Zeile 6 wird `var_pointer` als veränderbar deklariert und mit `Box::new(42)` ein Integer auf dem Heap alloziert. Zeile 7 übergibt `&mut var_pointer` an `increment`. Die Ownership von `var_pointer` bleibt im aufrufenden `main`, obwohl der Wert in der Funktion geändert wird.

```
1 fn increment(value: &mut i32) {
2     *value += 1;
3 }
4
5 fn main() {
6     let mut var_pointer = Box::new(42);
7     increment(&mut var_pointer);
8 }
9
```

Quelltext 4 : Borrowing mit veränderbarer Referenz [16]

Referenzen sind standardmäßig unveränderbar. Ein Aufruf wie `change(&s)` mit `&String` erlaubt nur Lesezugriff. Soll der Zielwert geändert werden, ist eine mutable Referenz nötig, erkennbar an `&mut` im Typ und beim Aufruf. Wie in Quelltext 4 muss `var_pointer` dann ebenfalls als `mut` deklariert sein. Damit Borrowing speichersicher bleibt, gelten zusätzliche Regeln [16]:

1. Zu jedem Zeitpunkt ist entweder genau eine veränderbare Referenz auf einen Wert erlaubt, oder beliebig viele unveränderbare Referenzen.
2. Referenzen müssen immer auf gültige Daten zeigen.

Die erste Regel lässt sich als Schreib-Lese-Trennung lesen. Existiert eine schreibende Referenz, darf keine zweite Referenz auf denselben Wert gleichzeitig aktiv sein, weder schreibend noch lesend. Existieren nur lesende Referenzen, sind mehrere parallel zulässig. Der Compiler lehnt Code ab, der zwei mutable Referenzen auf dieselbe Variable anlegt, oder der gleichzeitig liest und schreibt.

In nebenläufigen Programmen schützt dieselbe Logik vor Data Races, also gleichzeitigem Zugriff mehrerer Threads auf dieselben Daten, von dem mindestens einer schreibt. Data Races führen zu undefiniertem Verhalten. Rust erkennt viele davon bereits zur

Compilezeit. Eine vertiefte Betrachtung der Nebenläufigkeit liegt außerhalb des Fokus dieser Arbeit.

Borrowing wirft zunächst eine ähnliche Frage auf wie bei Zeigern in C. Wenn nur ausgeliehen wird, könnte eine Referenz theoretisch auf Daten zeigen, deren Owner längst nicht mehr existiert. Solche Dangling References wären Use After Free in neuer Form. Wie Rust das mithilfe des Borrow Checkers verhindert, wird im folgenden Abschnitt erläutert.

3.4 Lifetimes und Borrow Checker

Borrowing allein reicht nicht aus, um Dangling References auszuschließen. Entscheidend ist zusätzlich, wie lange eine Referenz gültig bleiben darf [16], [17]. Dafür führt Rust Lifetimes ein. Eine Lifetime beschreibt den Gültigkeitsbereich, in dem eine Referenz auf valide Daten zeigen darf. In den meisten Fällen leitet der Compiler Lifetimes automatisch ab. Werden Referenzen jedoch so verknüpft, dass die Gültigkeit unklar wäre, bricht die Kompilierung ab.

Der Borrow Checker ist die Compilezeitkomponente, die Ownership, Borrowing und Lifetimes gemeinsam prüft. Er vergleicht die Lebensdauer von Werten und Referenzen. Eine Referenz darf nicht länger existieren als der Wert, auf den sie zeigt. Verstöße werden als Kompilierfehler gemeldet, bevor ein Programm ausführbar entsteht. Quelltext 5 zeigt ein absichtlich fehlerhaftes Beispiel.

```
1  fn main() {
2      let reference;
3
4      {
5          let value = 5;
6          reference = &value;
7      }
8
9      println!("{}", reference);
10 }
11
```

Quelltext 5 : Ungültige Referenz über den Gültigkeitsbereich hinaus [17]

Der zeitliche Ablauf im Quelltext lässt sich entlanglesen. Drei Begriffe sollten dabei getrennt bleiben: die Variable `reference`, der Wert `value` und die Ausleihe, die `reference` von `value` übernimmt. Der Borrow Checker prüft, ob die Ausleihe nicht über das Ende von `value` hinausreicht [17].

Die Funktion `main` umschließt den gesamten Code. Gleich zu Beginn wird darin `reference` angelegt, zunächst noch ohne zugewiesenen Wert. Als Variable bleibt sie bis zum Ende der Funktion gültig. Solange sie uninitialisiert ist, enthält sie noch keine

Ausleihe. Ein innerer Block mit geschweiften Klammern eröffnet einen kürzeren Scope. Darin entsteht `value` mit dem Inhalt 5 auf dem Stack. `value` existiert nur, solange dieser Block aktiv ist. Beim Schließen des Blocks wird der Wert automatisch verworfen und der zugehörige Speicher freigegeben.

Noch innerhalb des Blocks wird `reference` über `&value` eine Referenz auf `value` zugewiesen. In diesem kurzen gemeinsamen Zeitraum sind Owner und Ausleihe gleichzeitig gültig. `value` besitzt den Wert 5, `reference` speichert dessen Adresse. Die Ausleihe ist deshalb gültig, weil der referenzierte Wert noch existiert. Sobald der innere Block endet, ist `value` nicht mehr vorhanden. Die Variable `reference` dagegen besteht weiter im äußeren Scope. Die zuvor gespeicherte Ausleihe darf jedoch nicht über das Ende von `value` hinaus gelten. Ab diesem Punkt würde `reference` auf bereits freigegebenen Speicher zeigen. Es entsteht eine Dangling Reference.

Am Ende der Funktion greift `println!` auf `reference` zu. Zu diesem Zeitpunkt existiert `value` nicht mehr. Der Zugriff würde Speicher lesen, der nicht mehr zum ursprünglichen Owner gehört. Das entspricht dem Use After Free Muster, hier jedoch über eine Referenz statt über einen rohen Zeiger.

Der Borrow Checker erkennt diesen Widerspruch zur Compilezeit. `reference` als Variable lebt länger als `value`. Die gültige Ausleihe in `reference` darf höchstens so lange gelten wie der Owner. Da die Referenz nach dem Verwerfen von `value` erneut genutzt wird, reicht die Ausleihe zu weit. Der Compiler bricht mit einem Fehler ab und verhindert die Erstellung eines ausführbaren Programms. Damit lassen sich Dangling References zur Compilezeit ausschließen, solange der Code den Regeln von Ownership und Borrowing folgt. Im folgenden Abschnitt wird gezeigt, wie dieselben Mechanismen konkret einen Use After Free Fehler verhindern, wie er zuvor in C auftrat.

3.5 Verhinderung von Use After Free

Die bisherigen Konzepte lassen sich an dem Heap Beispiel aus Quelltext 1 nachvollziehen. Dort reserviert `malloc()` Speicher für einen Integer, ein zweiter Zeiger verweist per Shallow Copy auf dieselbe Adresse, und `free()` gibt den Block frei. Der anschließende Zugriff über den verbleibenden Zeiger ist ein Use After Free. Quelltext 6 baut dasselbe Muster in Rust nach. Ziel ist nicht ein lauffähiges Programm, sondern die Frage, ob der Compiler den Fehler zulässt.

```rust
fn main() {
    let buffer_owner = Box::new(42);

    let buffer_borrow = &buffer_owner;

    drop(buffer_owner);

    println!("{}", buffer_borrow);
}

```

Quelltext 6 : Use After Free Muster in Rust parallel zu Quelltext 1

Gleich zu Beginn wird mit `Box::new(42)` ein Integer auf dem Heap alloziert. `Box<T>` ist ein Smart Pointer, der einen Wert auf dem Heap hält und beim Verlassen des Scopes automatisch `drop()` aufruft. `buffer_owner` wird damit zum Owner des Heap Blocks. Für Typen mit fester Größe wie `i32` wäre ein Stack Wert oft ausreichend. `Box<T>` wird hier dennoch eingesetzt, um strukturell an Quelltext 1 anzuknüpfen, in dem ebenfalls Heap Speicher explizit reserviert wird.

Im C Beispiel entsteht der zweite Zugriffspfad durch eine Shallow Copy. `buffer_ptr2` zeigt auf dieselbe Adresse wie `buffer_ptr1`, ohne dass ein neuer Block reserviert wird. In Rust ist ein derartiger zweiter Owner für `Box<T>` nicht zulässig. Move Semantics verhindern, dass zwei Variablen gleichzeitig denselben Heap Block besitzen [15]. Stattdessen leiht `buffer_borrow` den Owner über `&buffer_owner` aus. Die Ownership bleibt bei `buffer_owner`, zusätzlich existiert eine unveränderbare Referenz [16].

Als Nächstes soll der Heap Block freigegeben werden. In C entspricht das dem Aufruf von `free()`. In Rust würde `buffer_owner` am Ende seines Scopes ohnehin automatisch verworfen. Zur Verdeutlichung wird `drop(buffer_owner)` explizit aufgerufen. Der Aufruf beendet die Ownership und würde den Heap Speicher freigeben. Typischerweise wird `drop()` in Rust nicht von Hand aufgerufen, weil der Compiler die Freigabe an den passenden Stellen einfügt. Hier dient der explizite Aufruf als bewusste Parallele zu `free()` in Quelltext 1. Entscheidend ist, dass `drop()` die Besitzrechte verwirft und damit einen Move darstellt.

Anschließend soll über `buffer_borrow` auf den Wert zugegriffen und mit `println!` ausgegeben werden. Entspräche der Ablauf dem C Programm, läge ein Zugriff auf bereits freigegebenen Speicher vor. `buffer_borrow` würde dann auf einen Block zeigen, dessen Owner nicht mehr existiert, ein Use After Free Zugriffsfehler würde vorliegen.

Im Gegensatz zu C kompiliert Quelltext 6 nicht. Es entsteht kein ausführbares Programm, das den Fehler zur Laufzeit erst sichtbar macht. Der Borrow Checker bricht die Kompilierung bereits vor der Ausführung ab. Compilerausgabe 1 zeigt die zugehörige Fehlermeldung.

```
error[E0505]: cannot move out of `buffer_owner` because it is borrowed
 --> main.rs:6:10
  |
2 |     let buffer_owner = Box::new(42);
  |         ------------ binding `buffer_owner` declared here
3 |
4 |     let buffer_borrow = &buffer_owner;
  |                         ------------- borrow of `buffer_owner` occurs here
5 |
6 |     drop(buffer_owner);
  |          ^^^^^^^^^^^^ move out of `buffer_owner` occurs here
7 |
8 |     println!("{}", buffer_borrow);
  |                    ------------- borrow later used here
  |
help: consider cloning the value if the performance cost is acceptable
  |
4 |     let buffer_borrow = &buffer_owner.clone();
  |                                       ++++++++

For more information about this error, try `rustc --explain E0505`.
```

Compilerausgabe 1 : Kompilierfehler E0505 für Quelltext 6

Der Compiler meldet `error[E0505]: cannot move out of `buffer_owner` because it is borrowed`. Drei Stellen werden im Quelltext markiert. Beim Anlegen von `buffer_borrow` entsteht eine Ausleihe auf `buffer_owner`. Der anschließende Aufruf von `drop(buffer_owner)` würde den Owner bewegen und den Heap Block freigeben. Später soll `buffer_borrow` erneut verwendet werden. Solange die Ausleihe aktiv ist, darf `buffer_owner` nicht verschoben oder verworfen werden.

Damit greift dieselbe Logik wie im Lifetime Beispiel zuvor, hier jedoch auf Heap Daten mit `Box<T>`. Die Ausleihe in `buffer_borrow` darf nicht über das Ende von `buffer_owner` hinausreichen [17]. Der Compiler erkennt den Konflikt bereits am Aufruf von `drop()` und nicht erst beim späteren `println!`.

Die Fehlermeldung schlägt als mögliche Anpassung `buffer_owner.clone()` vor. Damit entstünde ein Deep Copy auf dem Heap, wie in Abbildung 4 dargestellt. Der Compiler weist zugleich auf den zusätzlichen Ressourcenaufwand hin (`if the performance cost is acceptable`). Ein solche Kopie wäre ein bewusster Ausweg, kein automatisches Verhalten wie das Shallow Copy in C.

Damit schließt Rust Use After Free in dieser Form bereits zur Compilezeit aus, solange die Ownership Regeln eingehalten werden. Im Gegensatz zu Laufzeitwerkzeugen wie AddressSanitizer [13] entsteht erst überhaupt kein fehlerhaftes Binary. Im nächsten Abschnitt wird betrachtet, wie Rust auch Out of Bounds Zugriffe behandelt.

3.6 Grenzprüfung zur Compilezeit

Neben Use After Free zählt Out of Bounds Write zu den zentralen Speicherzugriffsfehlern aus Tabelle 1 [10]. In Quelltext 2 wird der Fehler in C gezeigt. Dort schreibt `write_arr_at_index(array, 2)` außerhalb des gültigen Arraybereichs und trifft damit

benachbarten Stack Speicher, wie Abbildung 3 veranschaulicht. In C bleibt das Verhalten undefiniert und das Programm kompiliert trotzdem.

Rust unterscheidet zwei Fälle. Liegt der verwendete Index bereits zur Compilezeit fest, kann der Compiler Grenzverletzungen oft direkt erkennen. Quelltext 7 baut das Muster aus Quelltext 2 ohne Funktionsaufruf nach. Die Struktur bleibt gleich: eine lokale Variable `var`, ein Array mit zwei Elementen und ein Schreibzugriff mit Index 2.

```
1 fn main() {
2     let var = 5;
3     let mut array = [2, 2];
4
5     array[2] = 99;
6     println!("{}", var);
7 }
8
```

Quelltext 7 : Out of Bounds Write mit konstantem Index in Rust parallel zu Quelltext 2

`var` wird mit dem Wert 5 angelegt. Das Array `array` hat den Typ `[i32; 2]` und damit genau zwei Elemente. Gültige Indizes reichen von 0 bis 1. Der Ausdruck `array[2] = 99` verwendet dagegen den Index 2. Damit liegt derselbe logische Fehler vor wie in Quelltext 2, nur ohne separate Hilfsfunktion.

Entscheidend ist, dass sowohl die Arraylänge als auch der Index hier Konstanten sind. Der Compiler kennt beide Werte bereits beim Übersetzen des Programms. Er kann deshalb statisch prüfen, ob der Zugriff innerhalb der Arraygrenzen liegt. In Quelltext 2 wird der Index erst als Funktionsparameter übergeben. Für diesen direkten Zugriff mit konstantem Index reicht diese Information zur Compilezeit aus. Quelltext 7 kompiliert nicht. Compilerausgabe 2 zeigt die Fehlermeldung.

```
error: this operation will panic at runtime
 --> main.rs:5:5
  |
5 |     array[2] = 99;
  |     ^^^^^^^^ index out of bounds: the length is 2 but the index is 2
  |
  = note: `#[deny(unconditional_panic)]` on by default

error: aborting due to 1 previous error
```

Compilerausgabe 2 : Kompilierfehler für Quelltext 7

Der Compiler meldet `error: this operation will panic at runtime`. Markiert wird der Zugriff `array[2] = 99`. Durch die Meldung `index out of bounds: the length is 2 but the index is 2` wird deutlich, dass der Schreibzugriff bei jeder Ausführung außerhalb der Grenzen liegt. Standardmäßig wird dies über die Lint Regel

`unconditional_panic` abgelehnt, wie die Anmerkung `#[deny(unconditional_panic)] on by default` zeigt.

Damit entsteht kein ausführbares Programm, das den Fehler erst zur Laufzeit sichtbar macht. Der Out of Bounds Write aus Quelltext 2, der in C stillschweigend `var` überschreiben kann, wird bereits vor der Ausführung gestoppt. Im Gegensatz zu Use After Free, das der Borrow Checker über Ownership und Lifetimes verhindert, greift hier eine statische Analyse des Arrayzugriffs. Der Fall setzt voraus, dass Index und Arraygröße zur Compilezeit bekannt sind. Wird der Index erst zur Laufzeit übergeben, reicht diese Prüfung allein nicht aus.

3.7 Grenzprüfung zur Laufzeit

Quelltext 7 zeigte den einfachen Fall mit konstantem Index. Strukturell näher an Quelltext 2 liegt Quelltext 8. Dort erfolgt der Schreibzugriff in der Funktion `write_slice`, die ein Slice `&mut [i32]` und einen Index vom Typ `usize` entgegennimmt. Der Index wird beim Aufruf übergeben und ist für den Compiler zum Übersetzungszeitpunkt keine feste Konstante mehr. Ob `write_slice(&mut array, 2)` tatsächlich außerhalb der Grenzen liegt, lässt sich deshalb nicht mehr sicher zur Compilezeit entscheiden.

```
1  fn write_slice(arr: &mut [i32], index: usize) {
2      arr[index] = 99;
3  }
4
5  fn main() {
6      let var = 5;
7      let mut array = [2, 2];
8
9      write_slice(&mut array, 2);
10
11     println!("{}", var);
12 }
13
```

Quelltext 8 : Out of Bounds Write mit Laufzeitindex in Rust parallel zu Quelltext 2

In `main` entstehen wieder `var` mit dem Wert `5` und ein Array mit zwei Elementen. Der Aufruf `write_slice(&mut array, 2)` entspricht dem Aufruf von `write_arr_at_index(array, 2)` in Quelltext 2. Innerhalb von `write_slice` führt `arr[index] = 99` den eigentlichen Schreibzugriff aus. Anders als in Quelltext 7 kompiliert dieses Programm, weil der Index zur Compilezeit nicht als fest verletzt erkennbar ist.

Zur Laufzeit prüft Rust vor jedem Zugriff auf `arr[index]`, ob der Index innerhalb der Slice Grenzen liegt. Liegt er außerhalb, bricht das Programm kontrolliert ab, statt un-

definiertes Verhalten auszulösen. Rust kann diese Grenzverletzungen bei Arrayzugriffen zur Laufzeit erkennen, während die zentrale Speichersicherheit über Compilezeitprüfungen erreicht wird. Laufzeitmeldung 1 zeigt die Meldung bei Ausführung von Quelltext 8.

```
thread 'main' panicked at main.rs:2:5:
index out of bounds: the len is 2 but the index is 2
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

Laufzeitmeldung 1 : Panic Ausgabe für Quelltext 8

Der Laufzeitfehler beginnt mit `thread 'main' panicked at main.rs:2:5`. Als Ursache nennt die Meldung `index out of bounds: the len is 2 but the index is 2`. Der Panic tritt in `write_slice` auf, noch bevor der ungültige Schreibzugriff ausgeführt wird. `var` bleibt deshalb unverändert bei 5. In Quelltext 2 würde dagegen der Wert 99 ausgegeben, weil C den benachbarten Speicher stillschweigend überschreibt.

Diese Prüfung erfolgt automatisch. Der Rust Compiler fügt die nötigen Grenzchecks beim Übersetzen ein, ohne dass sie im Quelltext sichtbar werden müssen. Der zusätzliche Laufzeitaufwand bleibt gering, typischerweise genügen ein Vergleich und ein bedingter Sprung. Im Gegensatz zu nachträglicher Instrumentierung wie AddressSanitizer sind die Checks fester Bestandteil der normalen Programmausführung in Rust, nicht nur von Testbuilds.

Nach der Erkennung eines Out of Bounds Zugriffs übernimmt die Rust Laufzeitumgebung die Panic Behandlung. Standardmäßig ist `panic = "unwind"` gesetzt. Dabei wird der Stack abgebaut und der Prozess beendet. Mit `panic = "abort"` endet der Prozess ohne den Stack davor abzubauen [18]. In beiden Fällen wird der ungültige Speicherzugriff verhindert, bevor er ausgeführt wird. Das Verhalten ist damit vorhersehbar und nicht undefiniert wie in C.

Damit ergänzt die Laufzeitprüfung die Compilezeitanalyse aus dem vorherigen Abschnitt. Fehler mit konstantem Index werden bereits beim Kompilieren abgelehnt. Fehler mit dynamischem Index werden zur Laufzeit abgefangen. Zusammen mit Ownership, Borrowing und Lifetimes bildet Rust so mehrere Schutzschichten gegen die in Kapitel 2 betrachteten Speicherfehler.

3.8 Formale Sicherheitsnachweise

Die gezeigten Beispiele verdeutlichen das praktische Verhalten des Compilers und der Laufzeitumgebung. Offen bleibt, ob die zugrunde liegenden Regeln auch allgemein tragen. Laut [7] gilt eine Programmiersprache erst dann als speichersicher, wenn jedes in ihr erzeugbare Programm speichersicher ist. Rust erlaubt jedoch `unsafe` Blöcke, in denen typisierte Garantien bewusst außer Kraft gesetzt werden können, etwa für die Interaktion mit C Code oder für hardwarenahe Optimierungen [19].

Forschung adressiert diese Lücke zwischen Sprachversprechen und formaler Absicherung. [20] stellen mit RustBelt den ersten formalen und maschinell geprüften Sicherheitsbeweis für eine realistische Teilmenge von Rust vor. Die Autoren modellieren das ownershipbasierte Typsystem und zeigen, dass Programme innerhalb dieses Modells speichersicher bleiben. Entscheidend ist die Erweiterbarkeit des Beweises. Viele Standardbibliotheken nutzen intern `unsafe` Code. RustBelt formuliert dafür Verifikationsbedingungen, die eine Bibliothek erfüllen muss, damit sie als sichere Erweiterung der Sprache gilt. Für zentrale Bibliotheken des Rust Ökosystems wurde diese Prüfung durchgeführt [20].

Damit ist Rusts Speichersicherheit nicht nur eine Behauptung der Entwickler, sondern für einen klar abgegrenzten Kern wissenschaftlich untersucht. Der Beweis deckt nicht automatisch jedes Rust Programm ab, insbesondere wenn `unsafe` Code unsauber eingesetzt wird. Für den in dieser Arbeit betrachteten sicheren Teil der Sprache, der ohne `unsafe` auskommt, stützt RustBelt jedoch die Wirksamkeit der zuvor erläuterten Konzepte.

4 Fazit

Diese Arbeit hat Use After Free und Out of Bounds Write als zentrale Speicherzugriffsfehler betrachtet und Rusts Ansatz zur Compilezeitprüfung dagegen untersucht. Kapitel 2 zeigte, wie derartige Fehler in C entstehen, welche Sicherheitsfolgen drohen und welche alternativen Lösungsansätze sich etabliert haben. Kapitel 3 stellte dar, wie Ownership, Borrowing, Lifetimes und der Borrow Checker Dangling References verhindern, wie Use After Free bereits beim Kompilieren scheitert und wie Out of Bounds Zugriffe zur Compilezeit oder zur Laufzeit abgefangen werden.

Vorteile gegenüber manueller Speicherverwaltung in C liegen vor allem in der frühen Fehlererkennung. Viele Speicherfehler führen nicht zu lauffähigen Programmen, sondern zu Kompilierfehlern. Damit entsteht kein undefiniertes Verhalten zur Laufzeit. Rust verbindet diese Prüfungen mit direktem Speicherzugriff ohne Garbage Collector für den sicheren Kern der Sprache. Grenzverletzungen mit dynamischem Index werden zwar erst zur Laufzeit erkannt, aber kontrolliert über Panic abgebrochen, statt stillschweigend benachbarten Speicher zu überschreiben wie in Quelltext 2.

Nachteile und Grenzen sind ebenfalls zu berücksichtigen. Ownership, Borrowing und Lifetimes erfordern ein Umdenken und verlangsamen die Einarbeitung. Der Compiler lehnt Code ab, der in C noch kompiliert, was anfangs zusätzlichen Aufwand bedeutet. Rust ist nicht pauschal speichersicher, solange `unsafe` Code oder unsichere Schnittstellen zu C genutzt werden. Laufzeitprüfungen bei Arrayzugriffen erzeugen zudem einen geringen, aber messbaren Overhead. Für große bestehende C Codebasen bleibt ein vollständiger Sprachwechsel mit erheblichem Migrationsaufwand verbunden.

In der Praxis zeigt sich der Nutzen des Ansatzes. Google berichtet für Android, dass der Anteil speicherbezogener Schwachstellen an allen Sicherheitslücken von 76 Prozent im Jahr 2019 auf 24 Prozent bis Ende 2024 gesunken ist [21]. Hintergrund ist die Safe Coding Strategie, bei der neuer Code bevorzugt in speichersicheren Sprachen wie Rust entwickelt wird, ohne den gesamten bestehenden C und C++ Code ersetzen zu müssen. Abbildung 5 zeigt den absoluten Rückgang speicherbezogener Schwachstellen pro Jahr im selben Zeitraum.

Die Abbildung bestätigt den Trend der sinkenden absoluten Fallzahlen. Von etwa 223 speicherbezogenen Schwachstellen im Jahr 2019 geht der Wert bis 2024 auf etwa 36 zurück. Damit unterstreicht das Beispiel, dass Compilezeitprüfungen und speichersichere Sprachentwicklung nicht nur theoretisch wirksam sind, sondern in großen Produktionscodebasen messbare Sicherheitsgewinne erzielen können.

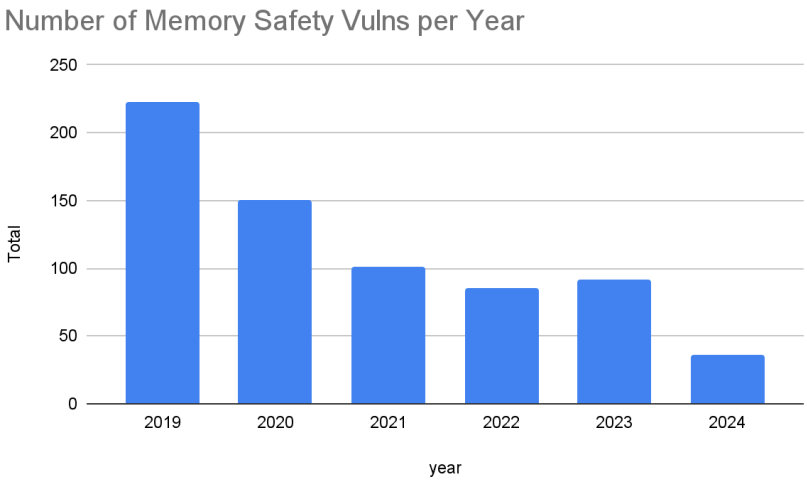


Abbildung 5 : Anzahl speicherbezogener Schwachstellen in Android pro Jahr [21]

Rust verhindert die in dieser Arbeit betrachteten Speicherfehler nicht in jedem Fall automatisch. Für den sicheren Teil der Sprache bietet es jedoch ein durchdachtes Regelwerk, das Fehler früh erkennt und damit genau jene Schwachstellen adressiert, die in der CWE Statistik zu den häufigsten und schwerwiegendsten zählen. Die Kombination aus Compilezeitprüfung und kontrollierter Laufzeitabsicherung macht Rust zu einer ernsthaften Alternative im Vergleich zu manueller Speicherverwaltung in C und zu nachträglicher Fehlersuche mit Laufzeitwerkzeugen allein.

Literaturverzeichnis

[1] "CWE - Common Weakness Enumeration." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/index.html

[2] "CWE - 2025 CWE Top 10 KEV Weaknesses." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_kev_list.html

[3] "CWE - 2024 CWE Top 10 KEV Weaknesses." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2024/2024_kev_list.html

[4] D. E. Mann and S. M. Christey, "Towards a Common Enumeration of Vulnerabilities."

[5] "CWE - 2025 CWE Top 25 Most Dangerous Software Weaknesses." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_methodology.html

[6] "CWE - 2025 CWE Top 10 KEV List Insights." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_kev_insights.html

[7] M. Hicks, "What is memory safety? - The PL Enthusiast." Accessed: May 30, 2026. [Online]. Available: http://www.pl-enthusiast.net/2014/07/21/memory-safety/

[8] L. Szekeres, M. Payer, T. Wei, and D. Song, "SoK: Eternal War in Memory," in *2013 IEEE Symposium on Security and Privacy*, May 2013, pp. 48–62. doi: 10.1109/SP.2013.13.

[9] "CWE - CWE-416: Use After Free (4.20)." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/416.html

[10] "CWE - CWE-787: Out-of-bounds Write (4.20)." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/787.html

[11] T. Baker, "Memory Safety: An Explainer." Accessed: May 30, 2026. [Online]. Available: https://cset.georgetown.edu/article/memory-safety-an-explainer/

[12] J. V. S.-A. team and A. R.-S. Foundations, "Eliminating Memory Safety Vulnerabilities at the Source." Accessed: May 30, 2026. [Online]. Available: https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html

[13] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A Fast Address Sanity Checker," 2012, pp. 309–318. Accessed: May 30, 2026.

[Online]. Available: https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany

[14] "Technology | 2025 Stack Overflow Developer Survey." Accessed: May 31, 2026. [Online]. Available: https://survey.stackoverflow.co/2025/technology/#2-programming-scripting-and-markup-languages

[15] "What is Ownership? - The Rust Programming Language." Accessed: May 31, 2026. [Online]. Available: https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html

[16] "References and Borrowing - The Rust Programming Language." Accessed: May 31, 2026. [Online]. Available: https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html

[17] "Validating References with Lifetimes - The Rust Programming Language." Accessed: May 31, 2026. [Online]. Available: https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html

[18] "Panic - The Rust Reference." Accessed: May 31, 2026. [Online]. Available: https://doc.rust-lang.org/reference/panic.html

[19] "Unsafety - The Rust Reference." Accessed: May 31, 2026. [Online]. Available: https://doc.rust-lang.org/reference/unsafety.html

[20] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "RustBelt: securing the foundations of the Rust programming language," *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, pp. 1–34, Jan. 2018, doi: 10.1145/3158154.

[21] J. V. S.-A. team and A. R.-S. Foundations, "Eliminating Memory Safety Vulnerabilities at the Source." Accessed: May 31, 2026. [Online]. Available: https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html